

Import Libraries

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Activation
from tensorflow.keras import regularizers
from tensorflow.keras.callbacks import EarlyStopping
import matplotlib.pyplot as plt
```

Load and Preprocess MNIST Dataset

```
# Load Dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize Pixel Values
x_train = x_train / 255.0
x_test = x_test / 255.0

# Flatten Images (28x28 → 784)
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)
```

Task 1: Simple Neural Network (No Regularization)

```
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history1 = model.fit(
    x_train,
    y_train,
    epochs=5,
    validation_data=(x_test, y_test)
)
```

```
print("Validation Accuracy:",  
      history1.history['val_accuracy'][-1])
```

Expected Result

Validation Accuracy $\approx$ 97%–98%

Task 2: Add L2 Regularization

```
model = Sequential([  
    Dense(  
        128,  
        activation='relu',  
        kernel_regularizer=regularizers.l2(0.01),  
        input_shape=(784,)  
    ),  
    Dense(  
        64,  
        activation='relu',  
        kernel_regularizer=regularizers.l2(0.01)  
    ),  
    Dense(  
        10,  
        activation='softmax',  
        kernel_regularizer=regularizers.l2(0.01)  
    )  
)  
)  
  
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)  
  
history2 = model.fit(  
    x_train,  
    y_train,  
    epochs=5,  
    validation_data=(x_test, y_test)  
)  
  
print("Validation Accuracy:",  
      history2.history['val_accuracy'][-1])
```

Observation

- Training accuracy is usually slightly lower.
 - Validation accuracy may improve if the original model was overfitting.
 - L2 regularization discourages excessively large weights.
-

Task 3: Add Dropout

```
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dropout(0.5),

    Dense(64, activation='relu'),
    Dropout(0.5),

    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history3 = model.fit(
    x_train,
    y_train,
    epochs=5,
    validation_data=(x_test, y_test)
)

print("Validation Accuracy:",
      history3.history['val_accuracy'][-1])
```

Observation

- Dropout randomly disables 50% of neurons during training.
 - Training accuracy decreases because the network learns under more challenging conditions.
 - Validation accuracy often improves because dropout reduces overfitting.
-

Task 4: Add Batch Normalization

```
model = Sequential([

    Dense(128, input_shape=(784,)),
    BatchNormalization(),
    Activation('relu'),

    Dense(64),
    BatchNormalization(),
    Activation('relu'),

    Dense(10, activation='softmax')

])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history4 = model.fit(
    x_train,
    y_train,
    epochs=5,
    validation_data=(x_test, y_test)
)

print("Validation Accuracy:",
      history4.history['val_accuracy'][-1])
```

Plot Accuracy Curves

```
plt.plot(history4.history['accuracy'], label='Train')
plt.plot(history4.history['val_accuracy'], label='Validation')

plt.title("Accuracy Curve")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

Observation

- Training becomes more stable.

- Validation accuracy often improves.
 - Batch Normalization helps the network converge faster and reduces sensitivity to weight initialization.
-

Task 5: Early Stopping

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=2,
    restore_best_weights=True
)

model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history5 = model.fit(
    x_train,
    y_train,
    epochs=20,
    validation_data=(x_test, y_test),
    callbacks=[early_stop]
)

print("Training Stopped After:",
      len(history5.history['loss']),
      "epochs")

print("Final Validation Accuracy:",
      history5.history['val_accuracy'][-1])
```

Observation

- Training stops automatically when the validation loss has not improved for **2 consecutive epochs**.

- This saves training time and helps prevent overfitting by keeping the best-performing model.